

SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 - CONCEPT MAPPING

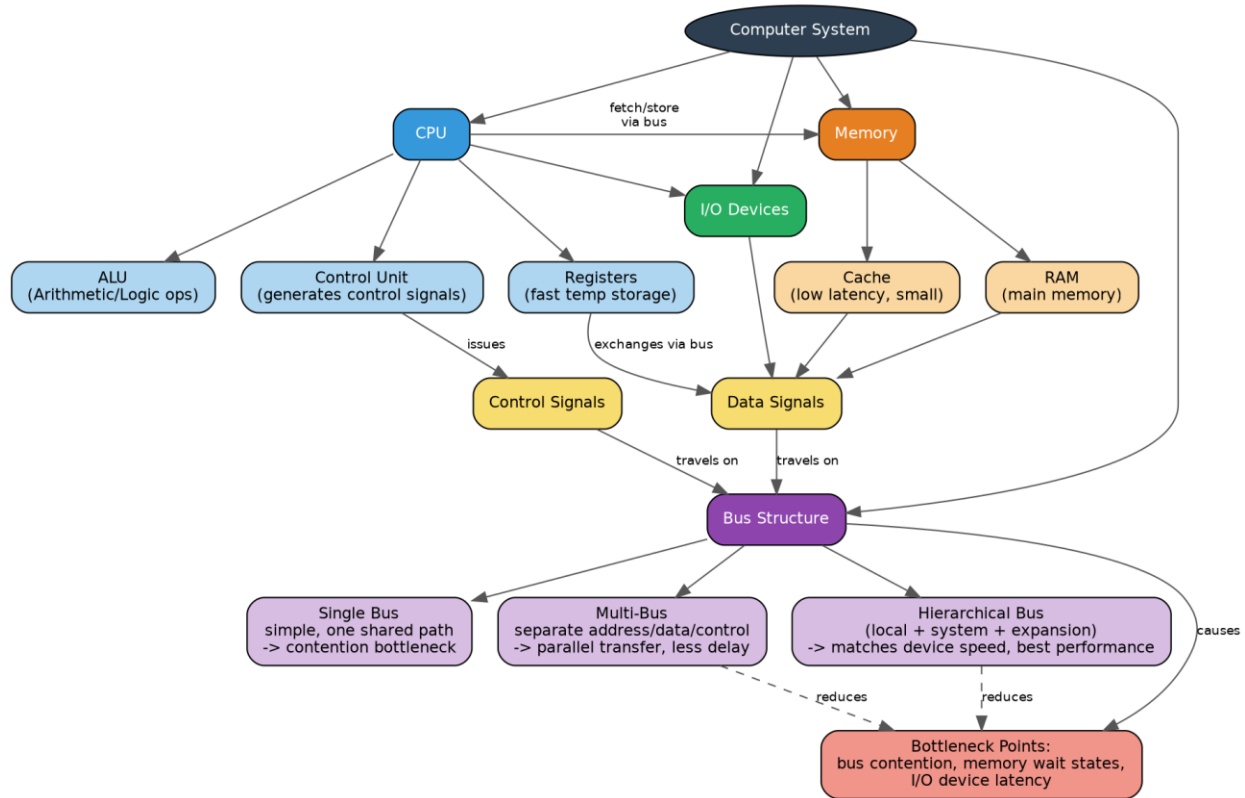
NAME=M.Dinesh kumar

Regno:192525298

CO1 Statement: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)

Q1. CPU, Memory, I/O and Bus Structures

Concept Map 1: CPU, Memory, I/O & Bus Structures

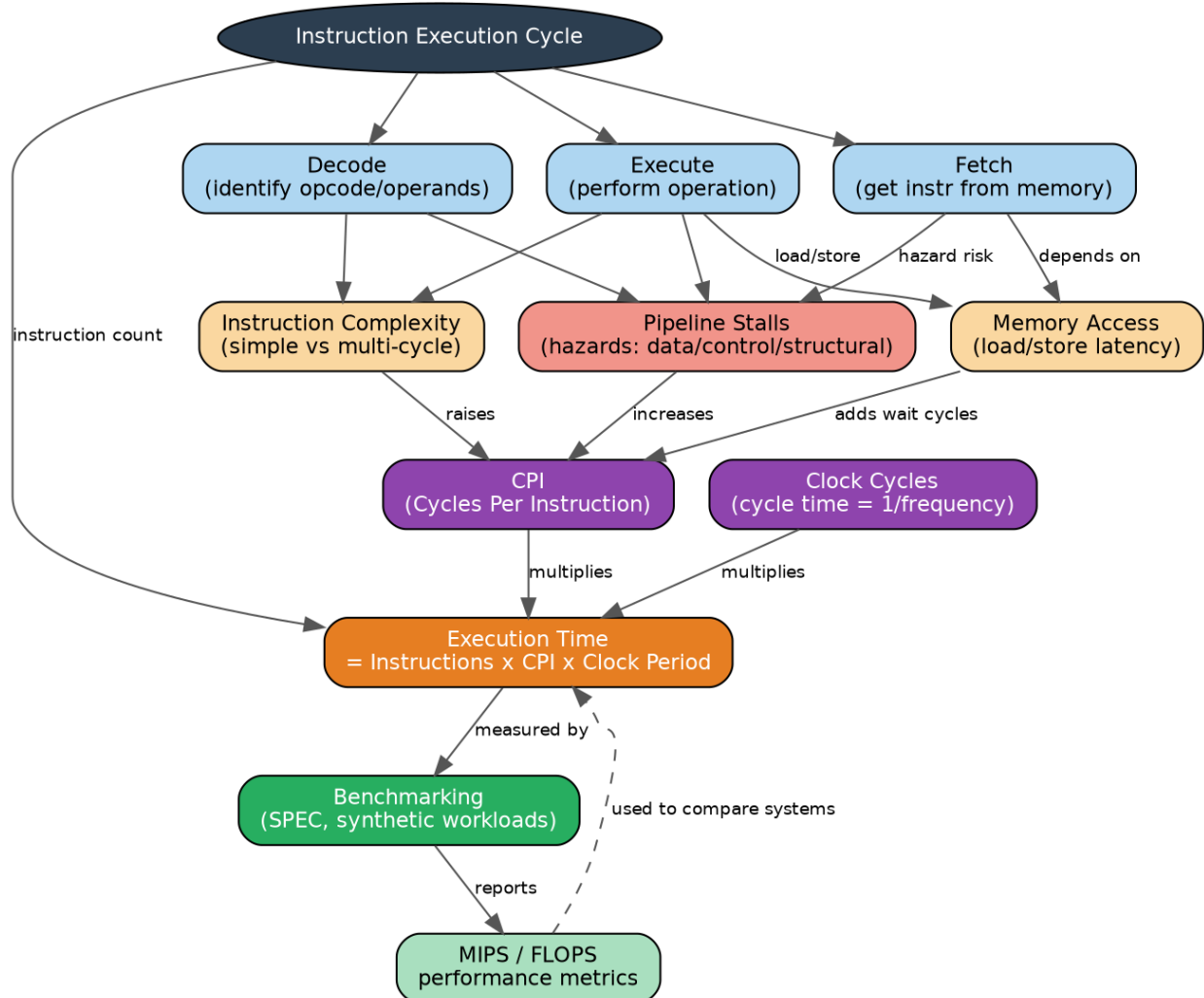


Notes:

- The CPU (ALU, Control Unit, Registers), memory (cache, RAM) and I/O devices are all connected through the system bus, which carries data and control signals.
- Single-bus designs are simple but create contention when multiple units need the bus at once, causing delay.
- Multi-bus designs separate address, data and control paths, allowing more parallel transfers and reducing bottlenecks.
- Hierarchical bus structures (local, system and expansion buses) match fast and slow devices to the right bus level, giving the best overall performance.
- Bottlenecks mainly arise from bus contention, memory wait states and slower I/O device response times.

Q2. Instruction Execution Cycle and CPU Performance

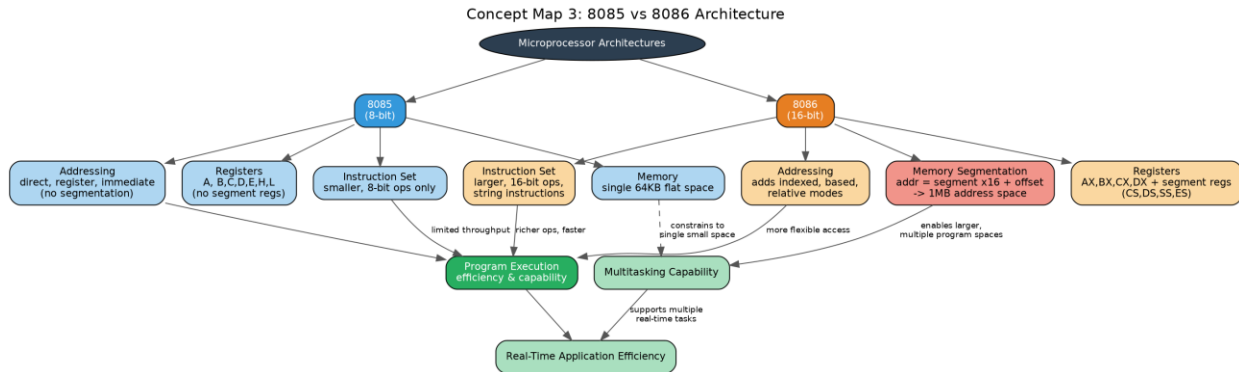
Concept Map 2: Instruction Cycle & CPU Performance



Notes:

- Every instruction passes through fetch, decode and execute stages; each stage consumes clock cycles.
- Memory access delays, instruction complexity and pipeline hazards (data, control, structural) all raise the effective CPI.
- Execution Time = Instruction Count x CPI x Clock Cycle Time — the fundamental CPU performance equation.
- Benchmarking (e.g., SPEC suites) runs standard workloads to measure real execution time and derive comparative metrics like MIPS/FLOPS.
- Pipeline stalls are a key reason real CPI is higher than the ideal CPI of a non-pipelined design.

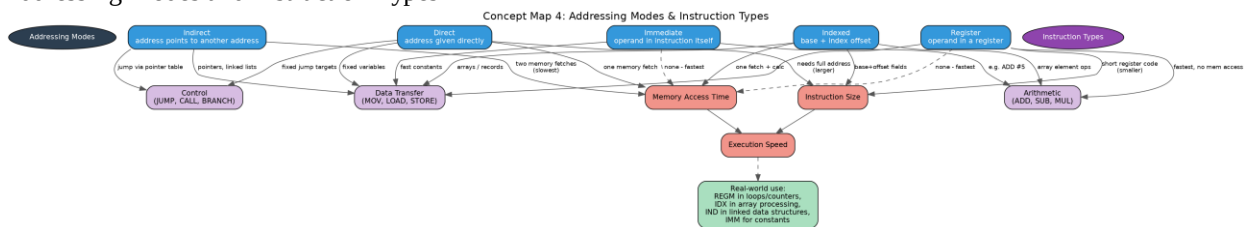
Q3. 8085 vs 8086 Architecture Comparison



Notes:

- The 8085 is an 8-bit processor with a simple register set and no memory segmentation, limiting it to a 64KB flat address space.
- The 8086 is 16-bit, adds segment registers (CS, DS, SS, ES) and uses segmentation (segment x 16 + offset) to address 1MB of memory.
- 8086's richer instruction set (string instructions, more addressing modes) and larger address space enable better multitasking support.
- These architectural differences make the 8086 more efficient for real-time, multi-program environments, while the 8085 suits simple, single-task embedded control.

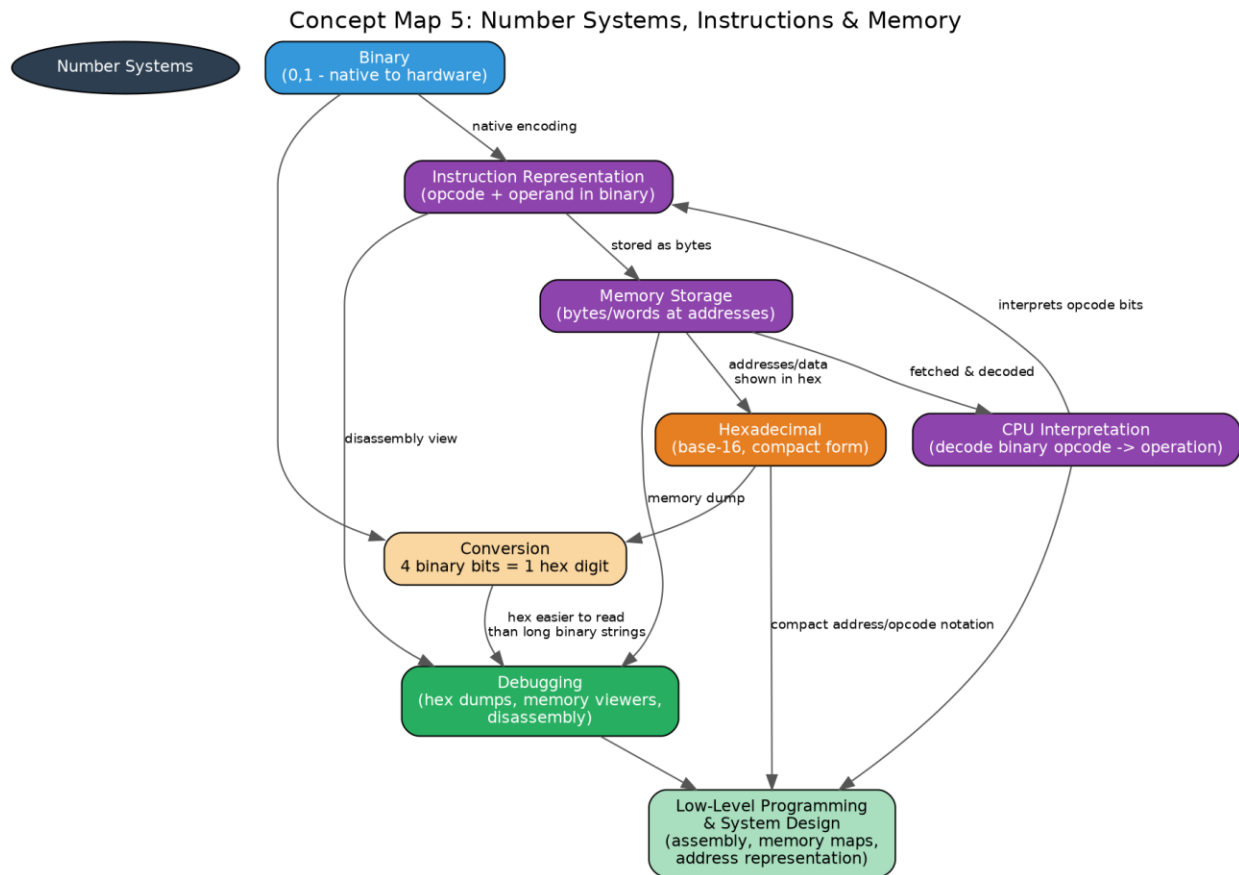
Q4. Addressing Modes and Instruction Types



Notes:

- Immediate and register addressing avoid memory access entirely, making them fastest and most compact — ideal for constants and loop counters.
- Direct addressing needs one memory fetch and a full address field, increasing instruction size.
- Indirect addressing needs two memory fetches (pointer, then data) — slowest, but essential for linked lists and pointer-based structures.
- Indexed addressing (base + offset) suits array and record processing, trading a small calculation for flexible access.
- Choice of addressing mode is therefore a direct trade-off between instruction size, memory access time and execution speed.

Q5. Number Systems, Instruction Representation and Debugging



Notes:

- Binary is the native language of hardware; every instruction and data value is ultimately stored as bits in memory.
- Hexadecimal is a compact, human-readable shorthand for binary (4 bits = 1 hex digit), making addresses and opcodes easier to read and write.
- Instructions are stored in memory as binary opcode + operand patterns, which the CPU fetches and interprets during decode.
- Hex is used pervasively in debugging (memory dumps, disassembly, register views) because it compresses long binary strings without losing exactness.
- This is why low-level programming and system design (assembly language, memory maps) rely heavily on hexadecimal notation.

